



Sri Sivasubramaniya Nadar College of Engineering

Department of Computer Science and Engineering

Title

**Noise-Aware Geometry-Guided Graph
Neural Initialization for Variational
Quantum Eigensolvers**

REVIEW REPORT – II

Submitted by

R.K. Barath (3122235001029)

Nithyasree.K (3122235001092)

Madhangi Karimanal (3122235001073)

Under the guidance of

Dr. Sneka Nandhini

Academic Year: 2026–2027

Contents

Abstract	1
1 Introduction	2
1.1 Background	2
1.2 Motivation	3
1.3 Related work	4
1.4 Objectives	4
1.5 Scope and contributions of this review	5
2 First Review - Queries and Answers	6
3 Problem statement	7
3.1 Evaluation metrics	7
3.2 Research questions	8
4 Methodology	9
4.1 Architectural Design for Proposed System	9
4.2 Experimental design: the study ladder	9
4.3 Evaluation protocol and design rationale	10
4.4 Modules	10
4.4.1 Module 1: Data and Graph Construction	11
4.4.2 Module 2: Quantum Simulation and Noise Model	11
4.4.3 Module 3: GNN Initializer (Core Model)	11
4.4.4 Module 4: Label-free Training	12
4.4.5 Module 5: VQE Optimization and Evaluation	12
5 Implementation	13
5.1 Module 1: Data and Graph Construction	13
5.2 Module 2: Quantum Simulation and Noise Model	14
5.3 Module 3: GNN Initializer	16
5.4 Module 4: Label-free Training	17
5.5 Module 5: VQE Optimization and Evaluation	19
5.6 Engineering issues found and fixed	20

5.7	Compute and runtime	20
6	Results and Discussion	21
6.1	S1 vs. S2: vanilla VQE vs. VQE+QNGD	21
6.2	S3, noiseless: GNN initialization on top of QNGD	22
6.3	S3, noisy: initialization under device noise	22
6.4	Convergence analysis: why a lower start energy can hurt	23
6.5	Accuracy ceiling of the two-layer ansatz	24
6.6	Training dynamics	24
6.7	Answers to the research questions	25
6.8	Discussion	25
6.9	Status against objectives and future work	27
7	Conclusion	28

List of Figures

4.1	High-level architecture of the proposed system. Solid arrows: inference and evaluation. Dashed red arrow: the label-free training signal, i.e. the gradient of the energy after K unrolled optimizer steps, $\nabla_{\phi}E(\boldsymbol{\theta}_K)$, back-propagated into the GNN.	9
4.2	Study ladder. S1→S2 isolates the optimizer, S2→S3 the learned initialization; the matrix of S3 variants isolates training objective (rows) and noise awareness (columns).	10
4.3	Modules and their dependencies.	11
5.1	Placement of the noise channels (three qubits shown; the CZ ring continues over all neighbouring pairs and the wrap-around pair).	15
5.2	GNN initializer. Orange components form the hardware/noise branch, removed entirely in the noise-unaware variants (84,738 vs. 70,850 parameters). The decoder also receives each gate slot’s own ansatz embedding (skip connection, not drawn).	16
5.3	Unrolled training: the energy after K simulated optimizer steps is back-propagated into the GNN.	17
6.1	Median circuit executions to the target (log scale).	21
6.2	Overall circuit reduction of each initializer over VQE+QNGD from random starts.	23

List of Tables

1.1	Related work and its relation to this review.	4
5.1	Dataset used in this review (2-layer ansatz, $P = 4n$ parameters). . . .	13
5.2	Graph representations fed to the GNN.	14
5.3	Noise presets of the hardware graph.	14
5.4	Validation of the PyTorch simulator against PennyLane (worst case over five instance families, all noise presets and topologies).	16
5.5	GNN hyperparameters.	17
5.6	Training configurations of the five GNN variants (Adam, learning rate 10^{-3}).	18
5.7	Measured circuit executions per optimizer step.	19
5.8	Issues found during implementation and how they were resolved. . . .	20
5.9	Approximate wall-clock times of the main jobs.	20
6.1	Vanilla VQE (Adam, 2000 steps) vs. VQE+QNGD (200 steps) from identical random starts (474 paired runs). Median executions are over runs that reached the target.	21
6.2	Noiseless test set (158 instances): circuit reduction versus VQE+QNGD from random starts (positive = fewer executions). Random starts reach the target in 79% of instances.	22
6.3	Noisy test set (150 instances, 4–8 qubits, each with its own sampled noise): circuit reduction versus VQE+QNGD from random starts (random starts reach the target in 43% of runs).	22
6.4	Noisy test set by noise level and by hardware topology (circuit reduction versus random starts; number of instances in brackets).	23
6.5	Anatomy of noiseless VQE+QNGD runs (means over the 158 test instances, in the energy units of the Hamiltonians).	24
6.6	Selected training losses (GD family: raw $E(\theta_3)$; QNGD family: normalized validation loss).	25
6.7	Summary of findings.	25
6.8	Status of the objectives.	27

Abstract

The Variational Quantum Eigensolver (VQE) estimates ground-state energies with a parameterized quantum circuit tuned by a classical optimizer; on near-term hardware its cost is dominated by the number of quantum circuit executions. This review reports the complete experimental pipeline built since Review I and the resulting study on 158 held-out spin-model instances (Heisenberg XYZ, Fermi–Hubbard and transverse-field Ising, 4–12 qubits) from the VQEzy dataset. We first reproduce the dataset’s own *vanilla* VQE (Adam, 2000 steps) under exactly the same conditions as our method, with hardware-realistic circuit accounting, and establish VQE with Quantum Natural Gradient Descent (QNGD) from random starts as the baseline: QNGD reaches the target energy in 82% of runs versus 54%, using about 2% of the circuit executions of vanilla VQE when both succeed (94% fewer in total). We then train a three-graph GNN initializer (Hamiltonian, ansatz and hardware/noise graphs fused by cross-attention) to predict the starting parameters, label-free, by unrolling a few optimizer steps (gradient descent or QNGD), each in a noise-unaware and a fully noise-aware variant trained under a noise model derived from the hardware graph. To make this possible we implemented an exact, batched, differentiable density-matrix simulator validated against PennyLane to 10^{-12} . Without noise, only the gradient-descent-trained GNN (GNN-GD) improves on the QNGD baseline (5.8% fewer executions; 65% on Ising instances); under noise it degrades (−11.4%), while the QNGD-trained models (GNN-A +15.1%, GNN-B +14.9%) and the noise-aware GD model (GNN-GD-B +12.8%) improve. Training under noise turns the GD recipe from −15.8% into +12.8%, and noise awareness gives the largest gain at high noise (+19.8%). A convergence analysis shows that a lower starting energy is *anti*-correlated with the final result, explaining why short-horizon training objectives mislead. No single initializer yet wins in both regimes; a seed study and the geometry head are planned next.

Keywords: Variational Quantum Eigensolver; Quantum Natural Gradient; Graph Neural Network; Parameter Initialization; Noise-Aware Learning; Meta-Learning

1. Introduction

1.1 Background

Variational Quantum Eigensolver. VQE [1, 2] estimates the ground-state energy E_0 of a Hamiltonian H by preparing a parameterized state $|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle$ on a quantum device and minimizing, with a classical optimizer,

$$E(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | H | \psi(\boldsymbol{\theta}) \rangle \geq E_0. \quad (1.1)$$

The inequality (variational principle) makes the energy itself a label-free figure of merit: lower is always better.

Where the cost comes from. H is a weighted sum of Pauli strings, $H = \sum_k c_k P_k$. Terms that do not qubit-wise commute must be measured with separate circuits, so one energy evaluation costs G circuits (G = number of measurement groups). On hardware, gradients are obtained with the parameter-shift rule, which for rotation gates is exact:

$$\frac{\partial E}{\partial \theta_j} = \frac{1}{2} \left[E\left(\boldsymbol{\theta} + \frac{\pi}{2} \mathbf{e}_j\right) - E\left(\boldsymbol{\theta} - \frac{\pi}{2} \mathbf{e}_j\right) \right], \quad (1.2)$$

so one gradient step with P parameters costs $(2P+1)G$ circuit executions. The total number of executions until convergence, not the number of iterations, is therefore the meaningful cost on noisy intermediate-scale quantum (NISQ) hardware [3].

Quantum Natural Gradient. Plain gradient descent treats every parameter direction alike, although equal parameter changes can move the quantum state by very different amounts. QNGD [4] measures distances in state space with the Fubini–Study metric

$$F_{ij}(\boldsymbol{\theta}) = \text{Re} \langle \partial_i \psi | \partial_j \psi \rangle - \langle \partial_i \psi | \psi \rangle \langle \psi | \partial_j \psi \rangle \quad (1.3)$$

and updates

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta F(\boldsymbol{\theta}^{(t)})^+ \nabla E(\boldsymbol{\theta}^{(t)}), \quad (1.4)$$

($^+$: pseudo-inverse). The full F costs $\mathcal{O}(P^2)$ circuits, so in practice the *block-diagonal* approximation is used: within each layer of mutually commuting rotations with generators K_i , $F_{ij} = \langle K_i K_j \rangle - \langle K_i \rangle \langle K_j \rangle$ measured in the state just before that layer, which needs only a few extra circuits per step.

Why initialization matters. Both convergence speed and final accuracy depend on the starting point θ_0 . Random starts in deep or wide circuits land in flat regions (barren plateaus [5]) where gradients vanish and executions are wasted. A good θ_0 reduces the number of optimizer steps, and therefore the number of executions.

1.2 Motivation

GNN-based initializers such as Qracle [6] map a problem/ansatz graph to θ_0 but do not condition on hardware topology and noise, and are not trained for the optimizer that will actually be run. On real devices the best starting point can depend on which qubits are noisy and how far apart coupled qubits are. This project therefore investigates a three-graph, noise-aware initializer trained *through* the optimizer, and measures success by the quantity that matters on hardware: circuit executions to reach the ground state.

1.3 Related work

Table 1.1: Related work and its relation to this review.

Work	Approach	Relation to this work
Qracle [6]	GNN maps problem/ansatz graph to VQE parameters	Closest prior work; no hardware/noise graph, not trained through QNGD
Jain et al. [8]; QSeer [9]	GNN warm-starts for QAOA	Show cross-instance and cross-size generalization of GNN initializers (QAOA only)
Cheng et al. [10]	Clifford-circuit initialization	Non-learned alternative; no noise conditioning
Meta-VQE [11]	Network trained on the VQE energy of parameterized Hamiltonians	Precedent for label-free energy loss (our objective (a))
Verdon et al. [15]	Recurrent network proposes parameters along a trajectory	Precedent for trajectory losses (our pilot)
MAML [12]; Andrychowicz et al. [13]	Learn an initialization / optimizer by unrolling inner steps	Basis of our unrolled-optimizer objective
Wu et al. [14]	Short-horizon bias in meta-optimization	Explains the failure mode we observe with 3-step QNGD training
VQEZy [7]	Dataset of VQE instances and Adam trajectories	Source of instances and of the vanilla VQE method

1.4 Objectives

1. To reproduce the vanilla VQE procedure of the VQEZy dataset [7] under the same conditions as the proposed method and establish VQE+QNGD from random starts as the baseline, with hardware-realistic circuit-execution accounting.
2. To design and implement a three-graph GNN initializer $f_\phi(G_H, G_A, G_D) \rightarrow \theta_0$ (Hamiltonian, ansatz and hardware/noise graphs fused by cross-attention).
3. To train the initializer label-free by unrolling the optimizer (gradient descent or QNGD) and to quantify the effect of the training objective.
4. To build a noise model driven by the hardware graph and train noise-unaware and fully noise-aware variants, evaluated in both noiseless and noisy settings.

5. To compare all variants against the VQE+QNGD baseline on held-out instances, including an unseen qubit count, and to identify the components that actually contribute.

1.5 Scope and contributions of this review

The study covers spin-model Hamiltonians from VQEzy (XYZ, Fermi–Hubbard, transverse-field Ising), a 2-layer hardware-efficient CZ–RX–RY ansatz, and exact noiseless and noisy (density-matrix) simulation. The geometry head $\hat{F}_0$, adaptive refinement and the QAOA ablation proposed in Review I are not yet implemented (Section 6.9). The contributions reported here are:

- a like-for-like comparison of vanilla VQE and VQE+QNGD with device-level circuit accounting on 474 paired runs;
- a differentiable, batched density-matrix simulator that reproduces PennyLane’s QNGD update exactly, enabling training *through* noisy QNGD steps on a GPU;
- a hardware-graph-driven noise model in which topology physically matters (SWAP routing);
- a controlled 2×3 study of training objective versus noise awareness with five trained initializers, evaluated noiselessly and under noise;
- a convergence analysis explaining when learned starts help and when they hurt.

2. First Review - Queries and Answers

- **Query 1:** Run the vanilla VQE method of the source paper (VQEzy) under the same conditions as our proposed method, so that the two are directly comparable.

Answer: Done (Study S1, Sections 5.5 and 6.1). The VQEzy procedure (Adam, learning rate 10^{-3} , 2000 steps, $\theta_0 \sim \mathcal{N}(0, 1)$) was re-implemented in our pipeline and run under *exactly* the same conditions as every other study: the same 158 held-out test instances, the same 2-layer CZ–RX–RY ansatz, the same PennyLane simulator, the same exact ground energies, and the *same three random starting points per instance*, which are saved and reused by the VQE+QNGD baseline. Circuit executions are counted identically for every method with `qml.Tracker` at device level; since VQEzy’s simulator-only adjoint gradient does not exist on hardware, each vanilla step is charged the parameter-shift cost a device would pay. While doing this we found that VQEzy’s own stored trajectories were produced with a one-layer ansatz (Section 5.1), which is why the method was re-run rather than compared with stored values.

- **Query 2:** Define a proper evaluation metric, e.g. reduction in the number of circuits, or how often the ground state is reached.

Answer: Defined in Chapter 3 and used for every result:

- (i) *circuit executions to target*: device-level executions (gradient, metric-tensor and measurement-group circuits included) until the energy is within 1% of the best energy reached on that instance (also reported: within 0.05 of the exact ground energy);
- (ii) *circuit reduction* of a method versus the baseline, $1 - \sum_i \text{cost}_{\text{method},i} / \sum_i \text{cost}_{\text{baseline},i}$, where a run that never reaches the target is charged its full budget;
- (iii) *success rate*: fraction of runs that reach the target (how often the ground state is reached);
- (iv) *final energy gap* $E_{\min} - E_0$ to the exact ground energy;
- (v) all of the above in noiseless and noisy settings.

3. Problem statement

Given a Hamiltonian H on n qubits, the 2-layer CZ–RX–RY ansatz $U(\boldsymbol{\theta})$ with $P = 4n$ parameters, and a hardware description D (coupling map and per-qubit/per-coupler error rates), **learn**

$$f_\phi(G_H, G_A, G_D) \rightarrow \boldsymbol{\theta}_0 \in \mathbb{R}^{2 \times n \times 2} \quad (3.1)$$

such that VQE+QNGD started from $\boldsymbol{\theta}_0$ reaches the target energy with **fewer quantum circuit executions** than VQE+QNGD started from random parameters.

- **Inputs:** graphs G_H (qubits, Pauli couplings), G_A (gate slots, CZ ring and layer edges), G_D (physical qubits, coupling map, readout/gate errors, T_1).
- **Output:** $\boldsymbol{\theta}_0$ (one RX and one RY angle per qubit per layer).
- **Constraints:** no optimized-parameter labels (label-free training); prediction must not require running VQE on the target instance; must generalize to unseen Hamiltonians and an unseen qubit count (12 qubits, trained on ≤ 8).

3.1 Evaluation metrics

For instance i and run r , let $E_{i,r}^{(t)}$ be the energy before step t and c_i the measured executions per step. With E_i^* the lowest energy reached on instance i by any method compared, the target is $T_i = E_i^* + 0.01 |E_i^*|$, and

$$t_{i,r}^{\text{hit}} = \min\{t : E_{i,r}^{(t)} \leq T_i\}, \quad (3.2)$$

$$\text{cost}_{i,r} = \begin{cases} c_i t_{i,r}^{\text{hit}} & \text{if reached,} \\ c_i t_{\max} & \text{otherwise (full budget charged),} \end{cases} \quad (3.3)$$

$$\text{Circuit reduction}(M) = 1 - \frac{\sum_i \text{cost}_i^M}{\sum_i \overline{\text{cost}}_i^{\text{random}}}, \quad (3.4)$$

where $\overline{\text{cost}}_i^{\text{random}}$ averages the three random starts. We also report the *success rate* (fraction of runs for which t^{hit} exists), the stricter success rate at $E \leq E_0 + 0.05$, the *final energy gap* $\min_t E^{(t)} - E_0$, and win counts (instances where a method costs fewer executions than the random average).

3.2 Research questions

- RQ1.** How many circuit executions does QNGD save over the vanilla VQE of the source paper under identical conditions?
- RQ2.** Does a learned GNN initialization reduce executions further on top of QNGD?
- RQ3.** How does the training objective (start energy, unrolled GD, unrolled QNGD) change what the GNN learns?
- RQ4.** Does noise awareness (hardware graph input and training under noise) help under device noise?

4. Methodology

4.1 Architectural Design for Proposed System

Figure 4.1 shows the end-to-end system. A VQEzy instance is converted into three graphs; the GNN predicts θ_0 ; VQE then runs (QNGD) on a noiseless or noisy simulator whose noise is taken from the hardware graph. During training the same simulator is unrolled for K optimizer steps and the resulting energy is back-propagated into the GNN.

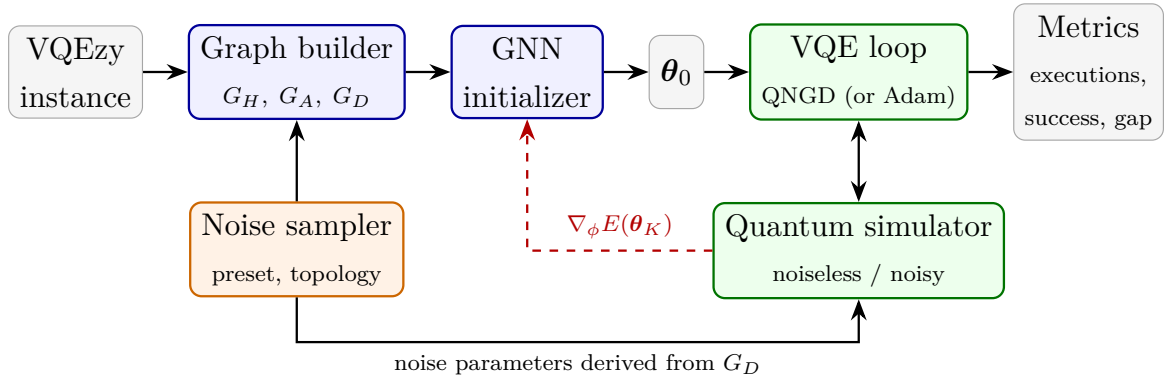


Figure 4.1: High-level architecture of the proposed system. Solid arrows: inference and evaluation. Dashed red arrow: the label-free training signal, i.e. the gradient of the energy after K unrolled optimizer steps, $\nabla_{\phi} E(\theta_K)$, back-propagated into the GNN.

4.2 Experimental design: the study ladder

The work is organised as a ladder of studies, each adding one ingredient, so that every gain can be attributed (Figure 4.2). All studies use the same 158 test instances, the same random starts and the same accounting.

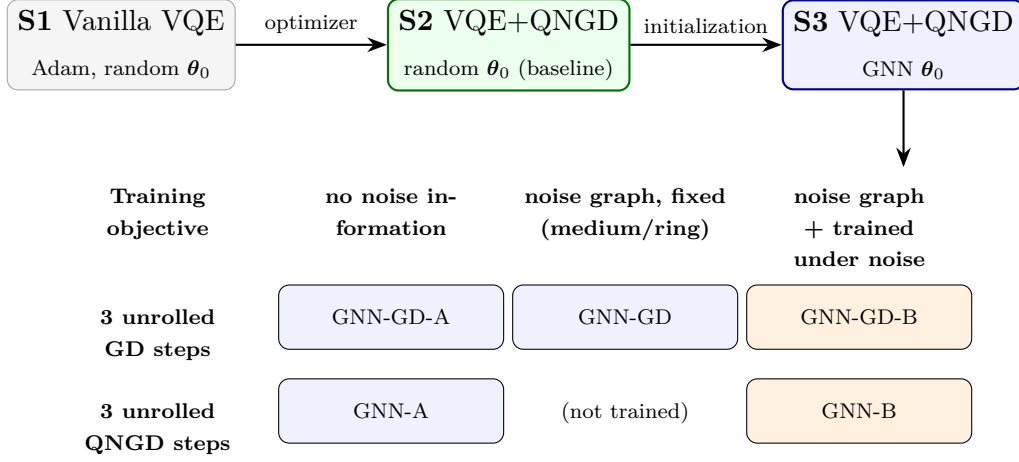


Figure 4.2: Study ladder. S1→S2 isolates the optimizer, S2→S3 the learned initialization; the matrix of S3 variants isolates training objective (rows) and noise awareness (columns).

4.3 Evaluation protocol and design rationale

- **Shared random starts.** Three random starts per instance are drawn once (fixed seed) and reused by vanilla VQE and VQE+QNGD, so that S1 versus S2 differs only in the optimizer.
- **Relative target.** With two layers the ansatz often cannot represent the exact ground state (Section 6.5); an absolute target would mark almost every run as failed and hide differences in speed. The relative target asks how fast each method reaches what the ansatz can achieve; the absolute success rate is reported alongside.
- **Budgets.** Vanilla VQE keeps VQEzy’s 2000 steps; QNGD runs 200 steps, which already exceeds the steps it needs on most instances while costing about one tenth of vanilla’s budget.
- **Held-out data.** All results are on the test split; model selection (the pilot) used the validation split only.
- **Unseen size.** The 12-qubit XYZ instances never appear in training; they test whether one set of GNN weights transfers to a larger circuit.

4.4 Modules

The implementation is split into five modules (Figure 4.3).

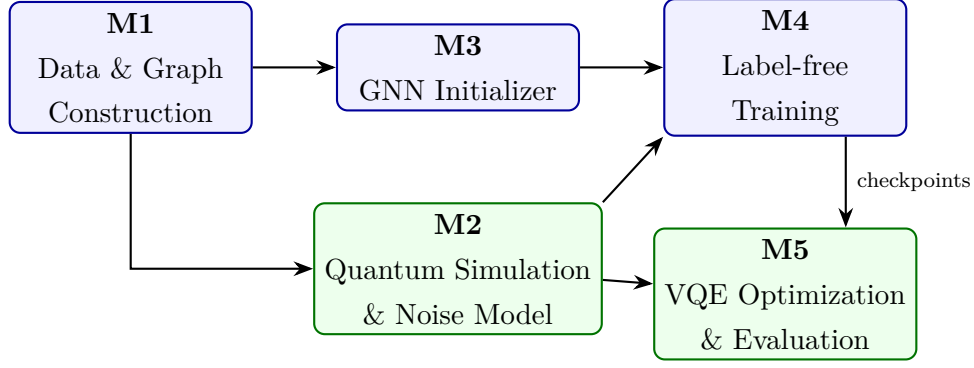


Figure 4.3: Modules and their dependencies.

4.4.1 Module 1: Data and Graph Construction

Purpose: turn dataset instances into model inputs and fixed experimental splits. **Inputs:** VQEzy .h5 files (couplings, reference trajectories). **Processing:** rebuild each Hamiltonian from its stored couplings; compute exact ground energies by diagonalization (≤ 12 qubits); build G_H , G_A , G_D ; persist a fixed train/validation/test split; derive a second, noise-varied copy of every split. **Outputs:** cached datasets `data_cache/` (fixed medium/ring hardware graph) and `data_cache_noisy/` (sampled noise preset and topology per instance). **Dependencies:** h5py, PennyLane, PyTorch Geometric.

4.4.2 Module 2: Quantum Simulation and Noise Model

Purpose: compute energies, gradients and QNGD steps, noiseless and noisy, fast enough to train through. **Inputs:** Hamiltonian matrix, θ , noise parameters derived from G_D . **Processing:** PennyLane (`lightning.qubit, default.mixed`) for reference evaluation and circuit counting; a custom batched PyTorch density-matrix simulator for training and noisy evaluation, validated against PennyLane. **Outputs:** $E(\theta)$, ∇E , block-diagonal metric F , unrolled trajectories. **Dependencies:** M1.

4.4.3 Module 3: GNN Initializer (Core Model)

Purpose: predict θ_0 from structure. **Inputs:** G_H, G_A, G_D . **Processing:** three GAT encoders, two-stage cross-attention fusion (ansatz $\rightarrow$ Hamiltonian, then $\rightarrow$ hardware), layer-wise decoder. A switch removes the hardware branch entirely (noise-unaware variants). **Outputs:** θ_0 . **Dependencies:** M1.

4.4.4 Module 4: Label-free Training

Purpose: fit the GNN without optimized-parameter labels. **Inputs:** training graphs, simulator (M2). **Processing:** unroll K steps of GD or QNGD from the predicted θ_0 and minimize the resulting energy; noisy variants simulate each instance under its own noise. **Outputs:** checkpoints of the five GNN variants. **Dependencies:** M2, M3.

4.4.5 Module 5: VQE Optimization and Evaluation

Purpose: run the baselines and the GNN-initialized VQE and score them. **Inputs:** test split, shared random starts, checkpoints. **Processing:** vanilla Adam VQE, VQE+QNGD (200 steps), device-level circuit counting, per-instance targets. **Outputs:** per-run trajectories (CSV) and comparison tables. **Dependencies:** M1–M4.

5. Implementation

All code is in the project repository (Python 3.13, PyTorch 2.8 with CUDA, PennyLane 0.45, PyTorch Geometric 2.6). Training and noisy evaluation ran on an NVIDIA RTX A5000 (24 GB); PennyLane evaluation ran on 18 CPU worker processes.

5.1 Module 1: Data and Graph Construction

Hamiltonians. Three spin-model families from VQEzy are used, rebuilt with PennyLane’s lattice constructors from the stored coupling constants:

$$H_{\text{XYZ}} = \sum_i \left(J_x X_i X_{i+1} + J_y Y_i Y_{i+1} + J_z Z_i Z_{i+1} \right), \quad J_x, J_y, J_z \in [-3, 3], \quad (5.1)$$

$$H_{\text{FH}} = -t \sum_{\langle ij \rangle, \sigma} \left(c_{i\sigma}^\dagger c_{j\sigma} + \text{h.c.} \right) + U \sum_i n_{i\uparrow} n_{i\downarrow}, \quad t, U \in [0, 5], \quad (5.2)$$

$$H_{\text{TI}} = -J \sum_{\langle ij \rangle} Z_i Z_j - h \sum_i X_i, \quad J, h \in [0, 5]. \quad (5.3)$$

XYZ is an open Heisenberg chain; Fermi–Hubbard is a chain of $n/2$ sites mapped to n qubits by the Jordan–Wigner transformation; the transverse-field Ising model lives on a 4×2 lattice. Exact ground energies E_0 are obtained by full diagonalization (up to 12 qubits).

Dataset and splits. Per file, 200/30/30 instances are drawn for train/validation/test with a fixed seed; the 12-qubit XYZ file is reserved for test only as an unseen-size check (Table 5.1).

Table 5.1: Dataset used in this review (2-layer ansatz, $P = 4n$ parameters).

Family	Hamiltonian	Qubits	Train	Val	Test
XYZ	Heisenberg chain	4	200	30	30
FH	Fermi–Hubbard, Jordan–Wigner	4, 6, 8	600	90	90
TI	Transverse-field Ising, 4×2	8	200	30	30
XYZ	Heisenberg chain (unseen size)	12	–	–	8
Total			1000	150	158

Graphs. Table 5.2 lists the node and edge features. The graph schemas are independent of n , so one model handles every size.

Table 5.2: Graph representations fed to the GNN.

Graph	Nodes	Node features	Edges and edge features
G_H	qubits	5 features describing the qubit’s terms in H	two-qubit Pauli couplings; 5 features (Pauli type, weight)
G_A	(qubit, layer) gate slots	4 features: qubit index, layer index, RX/RY parameter positions	CZ ring within a layer, same qubit across layers; one-hot type
G_D	physical qubits	readout error, $T_1/150\mu\text{s}$, single-qubit error	coupling map (linear, ring or grid); two-qubit error

Noise-varied dataset. For the noise study every instance of every split receives a new G_D with noise preset $\in \{\text{low, medium, high}\}$ and topology $\in \{\text{linear, ring, grid}\}$ sampled uniformly; all nine combinations occur in every split (88–134 training instances each). Preset base values, with $\pm 20\%$ per-qubit jitter, are given in Table 5.3.

Table 5.3: Noise presets of the hardware graph.

Preset	Readout error	1-qubit error	2-qubit error	T_1 (μs)
low	0.010	0.0005	0.003	150
medium	0.025	0.002	0.010	80
high	0.060	0.006	0.030	30

Data-integrity findings. (i) Fermi–Hubbard sample names repeat across the 4-, 6- and 8-qubit files, so instances must be keyed by (family, n , sample); an early script keyed by (family, sample) and was corrected. (ii) The parameters stored in VQEzy reproduce the stored final energies to 10^{-7} only with a *one-layer* ansatz, although the generating code defaults to two layers; the published trajectories are therefore one-layer results. This study keeps the two-layer ansatz throughout and re-runs the vanilla method instead of reusing stored numbers.

5.2 Module 2: Quantum Simulation and Noise Model

Ansatz. Each layer applies CZ on the ring $(0, 1), (1, 2), \dots, (n-1, 0)$, then $\text{RX}(\theta_{l,q,0})$ and $\text{RY}(\theta_{l,q,1})$ on every qubit q ; two layers give $P = 4n$ parameters (16 to 48).

Noise model. The noise acting on the circuit is computed from the numbers in G_D , so the model’s input and the simulated noise cannot disagree (Figure 5.1):

- after each $\text{CZ}(a, b)$: depolarizing channel on a and b with $p_{cz} = 1 - (1 - \bar{p}_{2q})^{1+6(d-1)}$, where d is the hop distance between a and b on the coupling map (each extra hop costs a SWAP there and back, i.e. six CNOT equivalents) and $\bar{p}_{2q}$ the mean coupler error along the path. Because the ansatz always uses a *CZ ring*, a linear chain pays heavily for the wrap-around CZ;
- after each RX/RY on q : depolarizing with the qubit's single-qubit error;
- at the end of each layer: amplitude damping $\gamma_q = 1 - e^{-t_{\text{layer}}/T_{1,q}}$ with $t_{\text{layer}} = (\text{CZ rounds}) \times 300 \text{ ns} + 2 \times 35 \text{ ns}$;
- before measurement: depolarizing with $p = 1.5 r_q$, which is exactly equivalent to a symmetric readout flip with probability r_q for Pauli expectation values (both scale each Pauli factor by $1 - 2r_q$).

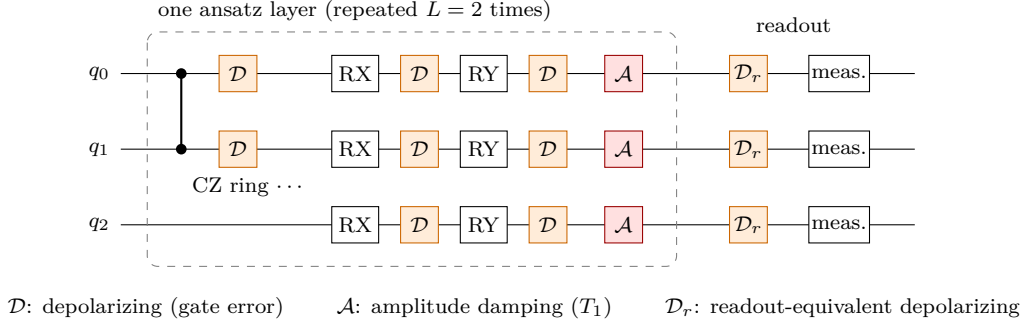


Figure 5.1: Placement of the noise channels (three qubits shown; the CZ ring continues over all neighbouring pairs and the wrap-around pair).

Differentiable density-matrix simulator. Training through QNGD steps needs $\partial\theta_K/\partial\theta_0$ through the metric tensor and its pseudo-inverse, and PennyLane's mixed-state device is far too slow for thousands of noisy instances. We therefore implemented an exact simulator in PyTorch (`src/quantum/torch_sim.py`) that evolves a batch of same-size density matrices $\rho \in \mathbb{C}^{B \times 2^n \times 2^n}$ on the GPU:

- one-qubit gates $\rho \mapsto U\rho U^\dagger$ via tensor contractions; CZ as a diagonal phase mask;
- depolarizing via $(1 - \frac{4p}{3})\rho + \frac{2p}{3} I_q \otimes \text{Tr}_q \rho$ (no Kraus sum needed); amplitude damping on the 2×2 sub-blocks;
- energy $\text{Tr}(\rho H)$; gradient by automatic differentiation;
- block-diagonal metric tensor exactly as PennyLane's `QNGOptimizer` builds it. PennyLane forms blocks greedily in gate order, which for this ansatz gives

$[RX_0], [RY_0, RX_1], \dots, [RY_{n-2}, RX_{n-1}], [RY_{n-1}]$ per layer (not “all RX, then all RY”); each block is the covariance of the generators $X_q/2$ or $Y_q/2$ in the state just before it. This detail was found only because the validation compared full QNGD trajectories, not just energies.

Table 5.4: Validation of the PyTorch simulator against PennyLane (worst case over five instance families, all noise presets and topologies).

Check	Worst discrepancy
Noiseless energy vs. <code>lightning.qubit</code>	4×10^{-16}
Noiseless QNGD trajectory (5 steps) vs. <code>QNGOptimizer</code>	8×10^{-5} rad
Noisy energy vs. <code>default.mixed</code>	9×10^{-12}
Noisy QNGD trajectory vs. <code>QNGOptimizer</code> on <code>default.mixed</code>	1×10^{-13}
Meta-gradient $\partial E(\boldsymbol{\theta}_3)/\partial \boldsymbol{\theta}_0$ vs. finite differences	3×10^{-8} (relative)
Single precision on GPU vs. double precision on CPU (3 steps)	2×10^{-6}

The small noiseless trajectory difference comes from near-zero eigenvalues of F , where the pseudo-inverse amplifies rounding differences; 10^{-4} rad over five steps is negligible.

5.3 Module 3: GNN Initializer

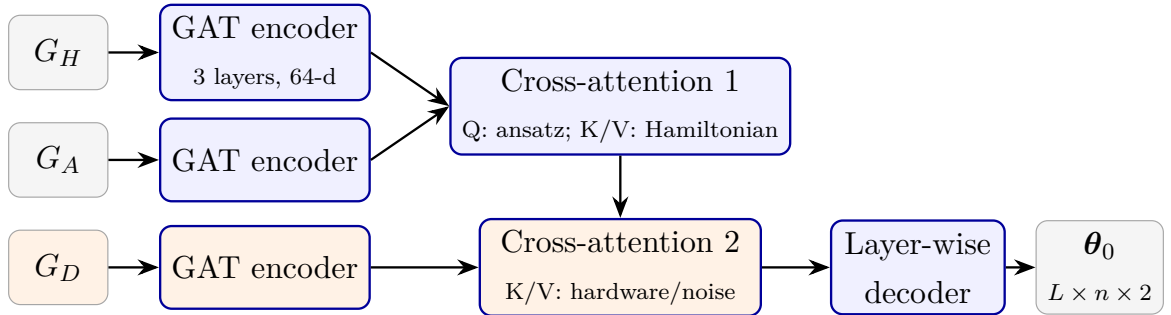


Figure 5.2: GNN initializer. Orange components form the hardware/noise branch, removed entirely in the noise-unaware variants (84,738 vs. 70,850 parameters). The decoder also receives each gate slot’s own ansatz embedding (skip connection, not drawn).

Each encoder is a 3-layer graph attention network [16] with residual connections and layer normalization, using edge features inside attention, implemented with PyTorch Geometric [20]. Fusion uses multi-head attention [17]: ansatz-node queries attend first to Hamiltonian nodes and then to hardware nodes, with a residual connection

and LayerNorm at each stage. The decoder maps each fused gate-slot embedding, concatenated with its ansatz embedding, to the slot’s two angles; the fixed node ordering reshapes directly to $\boldsymbol{\theta}_0 \in \mathbb{R}^{L \times n \times 2}$, so the same weights serve any qubit count. Table 5.5 lists the hyperparameters.

Table 5.5: GNN hyperparameters.

Component	Setting
Encoders	3 GAT layers, hidden size 64, 2 attention heads, residual + LayerNorm
Input dimensions	G_H : 5 node / 5 edge; G_A : 4 / 2; G_D : 3 / 1
Fusion	2 cross-attention stages, 4 heads, residual + LayerNorm
Decoder	MLP $128 \rightarrow 64 \rightarrow 2$ per gate slot (ReLU)
Parameters	84,738 (with G_D), 70,850 (without)
Optimizer	Adam, learning rate 10^{-3}

5.4 Module 4: Label-free Training

No optimized parameters are used as labels; the quantum objective itself is the loss. Three objectives were explored, each with precedent in learned initialization and meta-learning:

- (a) **Start energy**, $\mathcal{L} = E(\boldsymbol{\theta}_0)$, as in Meta-VQE [11]. Used by the first model; it produced low-energy starts in flat regions that converged *slower* than random starts (-41% under QNGD).
- (b) **Unrolled optimizer**, $\mathcal{L} = E(\boldsymbol{\theta}_K)$ with $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta G_k \nabla E(\boldsymbol{\theta}_k)$, where $G_k = I$ (GD, $\eta = 0.1$) or $G_k = F(\boldsymbol{\theta}_k)^+$ (QNGD, $\eta = 0.02$), and $K = 3$. This is MAML-style [12]: the start is judged by where the optimizer gets to.
- (c) **Trajectory mean**, $\mathcal{L} = \frac{1}{K} \sum_{k=1}^K E(\boldsymbol{\theta}_k)$ with $K = 15$, as in learned optimizers [13, 15]; tried as a pilot.

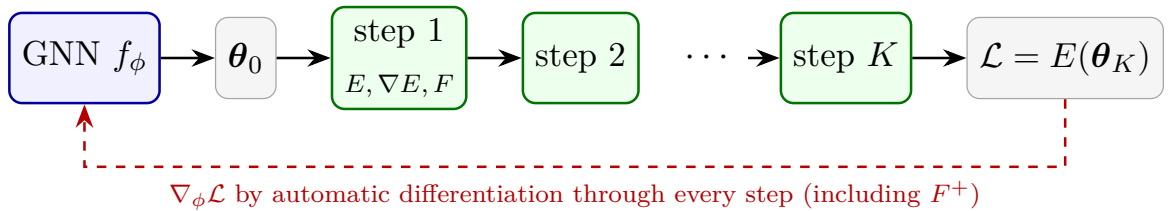


Figure 5.3: Unrolled training: the energy after K simulated optimizer steps is back-propagated into the GNN.

Algorithm 1: Label-free unrolled training of f_ϕ

Input: training instances $(G_H, G_A, G_D, H, \text{noise})$, steps K , step size η , optimizer type (GD or QNGD).

1. **for** each epoch, **for** each instance (or mini-batch of same-size instances):
2. $\theta_0 \leftarrow f_\phi(G_H, G_A, G_D)$
3. **for** $k = 0, \dots, K - 1$: simulate the circuit (with the instance’s noise if noise-aware), compute $E(\theta_k)$, ∇E (and F for QNGD), and set $\theta_{k+1} = \theta_k - \eta G_k \nabla E(\theta_k)$, keeping the computation graph
4. $\mathcal{L} \leftarrow E(\theta_K)$ (normalized by $\|H\|_1$ for the QNGD family)
5. back-propagate $\nabla_\phi \mathcal{L}$ through all K steps; Adam update of ϕ
6. keep the final epoch (GD family) or the best validation epoch (QNGD family)

Training correctness. The original unrolled training differentiated through parameter-shift gradients with PennyLane’s default `max_diff=1`, which silently returned about half the true meta-gradient (3.47 versus 6.80 on a test instance); it was replaced by exact backpropagation (Table 5.4).

Table 5.6: Training configurations of the five GNN variants (Adam, learning rate 10^{-3}).

Model	Objective	Noise in training	G_D input	Epochs	Checkpoint
GNN-GD	$E(\theta_3)$, 3 GD steps ($\eta = 0.1$), one instance per update	none	fixed medium/ring	12	last
GNN-GD-A	same as GNN-GD	none	removed	12	last
GNN-GD-B	same as GNN-GD	per instance, varied	varied	12	last
GNN-A	$E(\theta_3)/\ H\ _1$, 3 QNGD steps ($\eta = 0.02$), mini-batches	none	removed	30	best val.
GNN-B	same as GNN-A	per instance, varied	varied	30	best val.

The GD family copies the original GNN-GD recipe exactly (fixed order, one instance per update, final epoch kept) so that GNN-GD, GNN-GD-A and GNN-GD-B

differ only in noise awareness. The QNGD family uses GPU mini-batches of same-size instances and keeps the best validation epoch.

5.5 Module 5: VQE Optimization and Evaluation

Vanilla VQE (S1) reproduces VQEzy’s procedure: Adam [18], learning rate 10^{-3} , 2000 fixed steps, $\theta_0 \sim \mathcal{N}(0, 1)$. VQEzy used simulator-only adjoint gradients; on hardware each step needs parameter-shift, so each step is *charged* the measured parameter-shift cost. The trajectory is identical either way, since both give the exact gradient.

VQE+QNGD (S2, S3) uses PennyLane’s [19] `QNGOptimizer` (block-diagonal metric, $\eta = 0.02$, 200 steps). The *same* three random starts per instance are reused across S1 and S2, so the optimizer is the only difference.

Circuit accounting. Executions are counted with `qml.Tracker` at device level: a step costs $(2P + 1) \times G$ for parameter-shift, plus the metric-tensor circuits for QNGD (Table 5.7). Noisy runs use the PyTorch simulator and are charged the tracked cost of the same instance, since the circuits are identical.

Table 5.7: Measured circuit executions per optimizer step.

	XYZ-4	FH-4	FH-6	FH-8	TI-8	XYZ-12
Parameters P	16	16	24	32	32	48
Measurement groups G	3	5	5	5	3	3
Adam (parameter-shift)	99	165	245	325	195	291
QNGD (including metric)	105	170	251	341	213	317

QNGD’s block-diagonal metric adds only 3–9% per step, so any reduction in the number of steps translates almost one-to-one into fewer executions.

5.6 Engineering issues found and fixed

Table 5.8: Issues found during implementation and how they were resolved.

Issue	Effect	Resolution
Unrolled training used parameter-shift with <code>max_diff=1</code>	Meta-gradient about half its true value; training followed a wrong signal	Exact backpropagation; checked against finite differences
Sample names repeat across FH files	Different instances merged when keyed by (family, sample)	Key by (family, n , sample) everywhere
VQEzy stored data is one-layer	Stored energies not comparable with a two-layer ansatz	Re-run the vanilla method in our pipeline
PennyLane block-diagonal layering	First simulator version took different QNGD steps	Reproduced PennyLane’s greedy layering; trajectories now match to 10^{-13}
Thread oversubscription in parallel evaluation	About $12\times$ wasted CPU time	One thread per worker process

5.7 Compute and runtime

Table 5.9: Approximate wall-clock times of the main jobs.

Job	Hardware	Time
Vanilla VQE, 474 runs $\times$ 2000 steps	18 CPU workers	≈ 20 min
VQE+QNGD baseline, 474 runs $\times$ 200 steps	18 CPU workers	≈ 30 min
VQE+QNGD from one GNN, 158 runs	18 CPU workers	≈ 11 min
GD-family training, 12 epochs	GPU (one instance per update)	1–2 h
QNGD-family training, 30 epochs	GPU (mini-batches)	≈ 1 h
Noisy evaluation, 5 models + random	GPU	≈ 30 min
Simulator validation suite	CPU	< 1 min

6. Results and Discussion

6.1 S1 vs. S2: vanilla VQE vs. VQE+QNGD

Table 6.1: Vanilla VQE (Adam, 2000 steps) vs. VQE+QNGD (200 steps) from identical random starts (474 paired runs). Median executions are over runs that reached the target.

Group	Reached target		Median executions		QNGD cheaper (paired)	Total reduction
	Vanilla	QNGD	Vanilla	QNGD		
FH-4	54%	78%	253,275	8,663	70/74	94.0%
FH-6	50%	73%	395,430	21,627	66/72	93.2%
FH-8	46%	74%	569,725	21,266	67/71	93.8%
TI-8	94%	96%	280,800	3,408	86/90	97.6%
XYZ-4	40%	90%	145,778	6,431	81/83	94.7%
XYZ-12	8%	71%	391,541	31,066	17/17	92.2%
All	54%	82%	289,088	9,585	387/407	94.2%

QNGD reaches the target far more often and, where both succeed, with about 2% of the executions (median ratio 0.02). At QNGD’s whole budget ($\approx 44k$ executions) vanilla VQE is still far from the ground state (mean gap 10.4 versus 1.38). QNGD is *more* expensive per step (Table 5.7) but needs far fewer steps, so the total falls. Part of the gap reflects VQEzy’s small Adam learning rate (10^{-3}); a tuned first-order optimizer would narrow it, and our earlier gradient-descent runs with $\eta = 0.1$ were in the same range as QNGD. VQE+QNGD from random starts is therefore the baseline against which every initializer is judged.

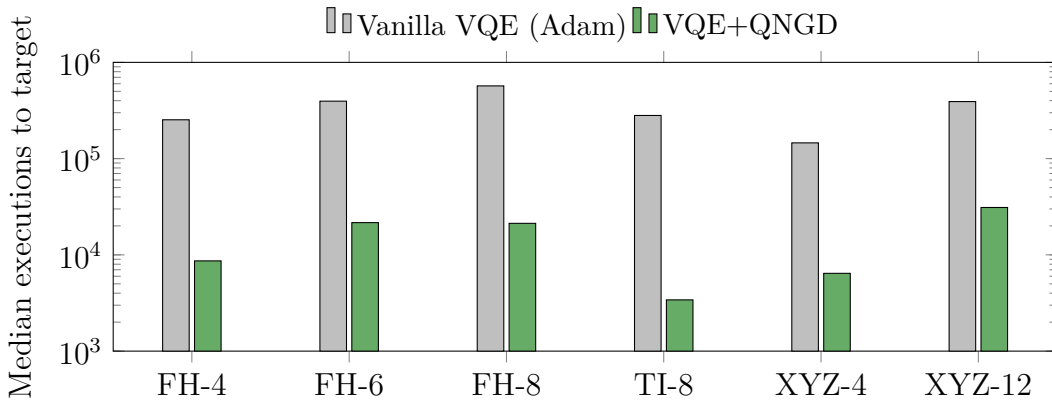


Figure 6.1: Median circuit executions to the target (log scale).

6.2 S3, noiseless: GNN initialization on top of QNGD

Table 6.2: Noiseless test set (158 instances): circuit reduction versus VQE+QNGD from random starts (positive = fewer executions). Random starts reach the target in 79% of instances.

Group	GNN-GD	GNN-GD-A	GNN-GD-B	GNN-A	GNN-B
FH-4	+15.4%	−38.9%	−65.3%	−26.8%	−67.5%
FH-6	−5.7%	−25.8%	−47.9%	−11.5%	−26.3%
FH-8	−7.6%	−71.8%	−68.3%	−64.9%	−77.7%
TI-8	+65.1%	+53.2%	−34.1%	+8.5%	−8.3%
XYZ-4	+12.9%	−0.9%	−14.3%	+2.2%	−26.4%
XYZ-12 (unseen size)	+24.3%	+35.3%	+41.6%	+47.5%	+7.7%
All	+5.8%	−29.7%	−43.7%	−22.8%	−44.8%
Reached target	80%	62%	57%	68%	52%
Wins vs. random	93/158	61/158	44/158	65/158	39/158

Only GNN-GD improves on the QNGD baseline without noise, driven by Ising (+65%) and the unseen 12-qubit size (+24%). Every variant generalizes to 12 qubits, which no model saw in training; this is the most consistent positive result of the study and supports the size-independent graph design. Fermi–Hubbard, especially with 8 qubits, is hard for all of them.

6.3 S3, noisy: initialization under device noise

Table 6.3: Noisy test set (150 instances, 4–8 qubits, each with its own sampled noise): circuit reduction versus VQE+QNGD from random starts (random starts reach the target in 43% of runs).

Group	GNN-GD	GNN-GD-A	GNN-GD-B	GNN-A	GNN-B
FH-4	+10.5%	−23.6%	−34.5%	−28.7%	−24.5%
FH-6	−2.7%	−3.9%	+1.0%	+7.1%	+11.6%
FH-8	−14.1%	−12.0%	−4.2%	−3.5%	−6.4%
TI-8	−35.6%	−35.6%	+92.2%	+92.0%	+94.4%
XYZ-4	−11.0%	−14.1%	+21.8%	+19.7%	+5.8%
All	−11.4%	−15.8%	+12.8%	+15.1%	+14.9%
Reached target	19%	12%	52%	53%	53%

Table 6.4: Noisy test set by noise level and by hardware topology (circuit reduction versus random starts; number of instances in brackets).

Subset	GNN-GD	GNN-GD-A	GNN-GD-B	GNN-A	GNN-B
high noise (59)	−6.8%	−7.0%	+13.8%	+13.3%	+19.8%
medium noise (51)	−12.4%	−19.4%	+12.4%	+16.3%	+11.8%
low noise (40)	−19.6%	−28.8%	+11.0%	+17.1%	+9.3%
linear (46)	−9.2%	−10.6%	+23.1%	+25.3%	+26.8%
ring (50)	−15.8%	−21.7%	+6.6%	+10.8%	+9.5%
grid (54)	−9.2%	−14.9%	+9.2%	+9.9%	+9.3%

All initializers that help under noise help most on the *linear* topology, where routing makes the wrap-around CZ very noisy and random starts suffer most; the noise-aware GNN-B is best there and at high noise, the two settings in which the hardware graph carries the most information.

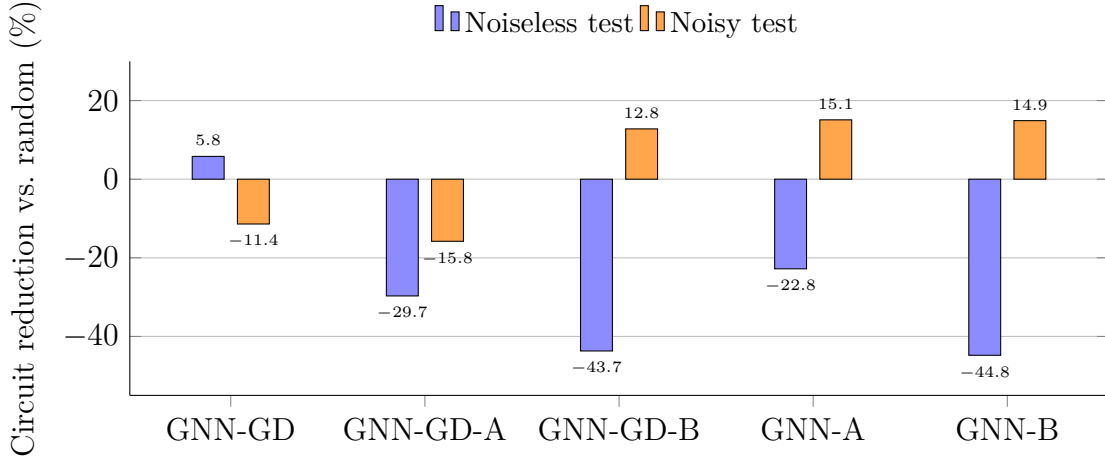


Figure 6.2: Overall circuit reduction of each initializer over VQE+QNGD from random starts.

6.4 Convergence analysis: why a lower start energy can hurt

Table 6.5 decomposes the noiseless QNGD runs into the energy gap at the start, the energy drop during the first three steps (the horizon the GNNs were trained on) and the drop afterwards.

Table 6.5: Anatomy of noiseless VQE+QNGD runs (means over the 158 test instances, in the energy units of the Hamiltonians).

Start	Gap at start	Drop, steps 1–3	Drop after step 3	Final gap	Reduction
Random	13.43	4.24	7.81	1.38	–
GNN-GD	10.85	4.95	4.61	1.30	+5.8%
GNN-GD-A	10.09	4.89	3.60	1.60	–29.7%
GNN-GD-B	8.79	4.34	2.88	1.58	–43.7%
GNN-A	7.07	3.11	2.45	1.52	–22.8%
GNN-B	6.69	2.30	2.61	1.78	–44.8%

The ordering is striking: the models with the *lowest* starting energies (GNN-B, GNN-A) have the *least* remaining descent and the worst final gaps, while GNN-GD, with the highest starting energy of the GNNs, ends best. Random starts are far from the target but sit on steep slopes and keep descending (7.81 after step 3). The GNNs learned starts that look good within their short training horizon and then stall in shallow basins, which is the short-horizon bias of meta-optimization [14]. Two consequences follow: the starting energy is a misleading quality measure for an initializer, and the training objective has to look further along the trajectory than three steps.

6.5 Accuracy ceiling of the two-layer ansatz

Independently of the initializer, only 13–16% of runs come within 0.05 of the exact ground energy (vanilla 14%, random starts with QNGD 16%, GNN variants 13–15%), and the mean final gap stays between 1.30 and 1.78. The two-layer ansatz itself therefore limits the attainable accuracy; initialization changes *how fast* the ansatz’s best energy is reached, not how good that energy is. This is why the primary metric uses a relative target, and it motivates deeper ansätze in future work.

6.6 Training dynamics

Training losses (Table 6.6) are comparable only within a family. GNN-GD-A tracks GNN-GD early but ends higher. Validation losses of the GD family swing by 10–40% between epochs (one instance per update), so “keep the last epoch” is partly luck; the QNGD family (mini-batches) moves by 1–5%. Lower training loss did not imply better VQE: GNN-A improved its loss the most yet is worse than random without noise, and the 15-step pilot reached a lower loss but scored –47% on the validation

set. This is consistent with the convergence analysis: the training losses measure a short horizon, while the evaluation measures the whole run.

Table 6.6: Selected training losses (GD family: raw $E(\boldsymbol{\theta}_3)$; QNGD family: normalized validation loss).

Model	Epoch 0	Middle	Final	Kept checkpoint
GNN-GD (train)	−8.50	−10.24 (ep. 4)	−10.07	last
GNN-GD-A (train)	−8.56	−9.90 (ep. 4)	−9.29	last
GNN-GD-B (val, noisy)	−4.56	−2.11 (ep. 6)	−4.71	last (= best)
GNN-A (val)	−0.300	−0.299 (ep. 10)	−0.321	best, −0.328 (ep. 24)
GNN-B (val, noisy)	−0.152	−0.155 (ep. 10)	−0.164	best, −0.165 (ep. 22)

6.7 Answers to the research questions

Table 6.7: Summary of findings.

Research question	Finding
RQ1: QNGD vs. vanilla VQE	94% fewer circuit executions; success rate 54%→82%; QNGD cheaper in 387 of 407 paired runs.
RQ2: GNN on top of QNGD	Noiseless: only GNN-GD helps (+5.8%; Ising +65%, unseen 12 qubits +24%). Noisy: GNN-A +15.1%, GNN-B +14.9%, GNN-GD-B +12.8%.
RQ3: Training objective	Start energy fails (−41%). Three GD steps give the best noiseless model; three QNGD steps learn starts that stall (short-horizon bias) but are robust to noise. A longer horizon without normalization was worse (−47%).
RQ4: Noise awareness	Training under noise is essential for the GD recipe (−15.8% → +12.8%; 70 vs. 16 instance wins). The hardware graph helps most at high noise (+19.8%) and on linear topologies, and slightly hurts at low noise.

6.8 Discussion

1. The optimizer matters most. Switching vanilla VQE to QNGD saves 94% of executions and raises the success rate from 54% to 82%. Learned initialization adds a smaller, regime-dependent improvement on top, so any claim about an initializer must be made against QNGD, not against vanilla VQE.

2. The training objective decides what a “good start” is. Minimizing the start energy produced starts that converge slowly. Unrolling three GD steps gave the only noiseless improvement. Unrolling three QNGD steps produced starts that descend quickly for a few steps and then stall (Table 6.5). Lengthening the horizon to 15 steps without normalization made it worse (-47%), most likely because the large Ising energies then dominated the loss.

3. Noise flips the ranking. The noiseless-trained GD models are noise-blind: GNN-GD’s predictions change by at most 6×10^{-4} rad whatever G_D it receives, and both it and GNN-GD-A fail under noise (Ising: 0 of 30 instances reached). Training the same recipe under noise (GNN-GD-B) turns -15.8% into $+12.8\%$ and wins 70 of 150 instances against GNN-GD-A (16 the other way), the clearest effect in the study. QNGD-trained models are robust to noise even without noise training (GNN-A $+15.1\%$).

4. Noise awareness pays off when noise is informative. GNN-B is best at high noise ($+19.8\%$, 49% versus 41% success) and on linear topologies, but worse at low noise. Depolarizing and readout noise mostly shrink the landscape towards zero without moving its minimum; only amplitude damping, heterogeneous error rates and routing shift the best start, so the hardware graph carries information mainly when these effects are strong.

5. Why Fermi–Hubbard is hard. FH instances have the most measurement groups (five) and, after the Jordan–Wigner mapping, long-range Pauli strings that a shallow nearest-neighbour ansatz represents poorly: even random starts with QNGD end with the largest gaps (1.78 and 2.67 for 6 and 8 qubits). Predicting a good start is hardest where the ansatz itself fits worst.

Comparison with existing work. Qracle [6] reports GNN initialization for VQE with gradient-based optimizers; to our knowledge, conditioning on a hardware/noise graph, training through differentiable QNGD steps and evaluating under graph-derived noise have not been combined before (to be confirmed by a full literature search). Our results also qualify the use of VQezy’s stored trajectories as baselines: they are one-layer runs and should be re-run under the conditions of any new method.

Limitations and threats to validity.

- One training seed per model; GPU evaluation varies by ≈ 0.5 points between reruns, and gaps of a few points between models are not significant.

- GNN-GD-A, intended as GNN-GD without the hardware branch, scores -29.7% versus $+5.8\%$ noiselessly; whether GNN-GD’s result is reproducible must be checked with several seeds.
- In the noiseless test, noise-aware models receive the dataset’s medium/ring graph while the circuit is noiseless; they never saw a zero-noise graph in training.
- The target is relative to the methods compared; the noise presets are synthetic rather than device calibration data; the 12-qubit noisy case was not simulated.

6.9 Status against objectives and future work

Table 6.8: Status of the objectives.

Objective	Evidence	Status
1. Vanilla and QNGD baselines	Table 6.1	Completed
2. Three-graph GNN initializer	Figure 5.2, five trained variants	Completed
3. Label-free unrolled training	Objectives (a)–(c), Table 6.5	Completed; horizon issue open
4. Noise model and noise-aware variants	Figure 5.1, Tables 6.3–6.4	Completed
5. Comparison and ablation	Tables 6.2–6.4	Partial (single seed)
Geometry head $\hat{F}_0$, adaptive refinement, QAOA ablation	–	Planned

Next steps:

- (i) a seed study (three seeds per model, keeping best and last epochs) to establish which gains are real;
- (ii) a zero-noise preset in noise-aware training so that one model serves both regimes;
- (iii) a normalized, longer-horizon objective to counter short-horizon bias;
- (iv) the geometry head $\hat{F}_0$ to reduce QNGD’s per-step metric cost;
- (v) the QAOA ablation and a Qracle-style baseline.

7. Conclusion

This review delivered a complete, validated pipeline for learned VQE initialization and a like-for-like comparison that answers the panel’s request to evaluate the source paper’s vanilla VQE under the same conditions as our method. Under identical instances, starts and circuit accounting, QNGD reduces circuit executions by 94% compared with vanilla VQE, which makes VQE+QNGD the baseline that any initializer must beat. On top of it, a GNN trained by unrolling gradient-descent steps reduces executions by a further 5.8% without noise, with large gains on Ising models and on an unseen qubit count, while QNGD-trained and noise-aware models reduce executions by 13–15% under device noise, up to 20% at high noise. The convergence analysis shows why the results depend on the regime: short training horizons reward starts that look good early and then stall. The immediate next steps are a multi-seed study to confirm these effects, noise-aware training that includes the noiseless case, a longer-horizon objective, and the geometry head proposed in Review I.

References

- [1] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O’Brien, “A variational eigenvalue solver on a photonic quantum processor,” *Nature Communications*, vol. 5, 4213, 2014. DOI: <https://doi.org/10.1038/ncomms5213>.
- [2] J. Tilly *et al.*, “The variational quantum eigensolver: A review of methods and best practices,” *Physics Reports*, vol. 986, pp. 1–128, 2022. DOI: <https://doi.org/10.1016/j.physrep.2022.08.003>.
- [3] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018. DOI: <https://doi.org/10.22331/q-2018-08-06-79>.
- [4] J. Stokes, J. Izaac, N. Killoran, and G. Carleo, “Quantum natural gradient,” *Quantum*, vol. 4, p. 269, 2020. DOI: <https://doi.org/10.22331/q-2020-05-25-269>.
- [5] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, “Barren plateaus in quantum neural network training landscapes,” *Nature Communications*, vol. 9, 4812, 2018. DOI: <https://doi.org/10.1038/s41467-018-07090-4>.
- [6] C. Zhang, L. Jiang, and F. Chen, “Qracle: A graph-neural-network-based parameter initializer for variational quantum eigensolvers,” *Proc. IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 197–207. DOI: <https://doi.org/10.1109/QCE65121.2025.00031>.
- [7] “VQEzy: A dataset for variational quantum eigensolver parameter initialization,” arXiv:2509.17322, 2025. Code and data: <https://github.com/chizhang24/VQEzy>.
- [8] N. Jain, B. Coyle, E. Kashefi, and N. Kumar, “Graph neural network initialization of quantum approximate optimisation,” *Quantum*, vol. 6, p. 861, 2022. DOI: <https://doi.org/10.22331/q-2022-11-17-861>.
- [9] L. Jiang, C. Zhang, and F. Chen, “QSeer: A quantum-inspired graph neural network for parameter initialization in quantum approximate optimization algorithm circuits,” *Proc. IEEE QCE*, 2025, pp. 1496–1504. DOI: <https://doi.org/10.1109/QCE65121.2025.00167>.

- [10] M. H. Cheng *et al.*, “Clifford circuit initialization for variational quantum algorithms,” *Physical Review A*, vol. 111, 062413, 2025. DOI: <https://doi.org/10.1103/PhysRevA.111.062413>.
- [11] A. Cervera-Lierta, J. S. Kottmann, and A. Aspuru-Guzik, “Meta-variational quantum eigensolver: Learning energy profiles of parameterized Hamiltonians for quantum simulation,” *PRX Quantum*, vol. 2, 020329, 2021. DOI: <https://doi.org/10.1103/PRXQuantum.2.020329>.
- [12] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” *Proc. International Conference on Machine Learning (ICML)*, 2017. arXiv:1703.03400.
- [13] M. Andrychowicz *et al.*, “Learning to learn by gradient descent by gradient descent,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1606.04474.
- [14] Y. Wu, M. Ren, R. Liao, and R. Grosse, “Understanding short-horizon bias in stochastic meta-optimization,” *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1803.02021.
- [15] G. Verdon *et al.*, “Learning to learn with quantum neural networks via classical neural networks,” arXiv:1907.05415, 2019.
- [16] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.
- [17] A. Vaswani *et al.*, “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.
- [18] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.
- [19] V. Bergholm *et al.*, “PennyLane: Automatic differentiation of hybrid quantum-classical computations,” arXiv:1811.04968, 2018.
- [20] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. arXiv:1903.02428.